

Artificial Intelligence

COURSE CODE: DS5216

NAME : A.M.T.K SRIMAL
REGISTRATION NUMBER : DTS2409
DATE OF SUBMISSION : 2026/06/30

In this assignment, I developed a computer vision system to track players in sports videos. Ten short video clips (each between five and ten seconds) were downloaded from YouTube as the dataset. Using Roboflow, I annotated video frames with two classes: player and non-player. The project uses a YOLO-style object detection approach to identify players and includes an optional Open Pose-like key point detection component for estimating player joints. Links to the dataset and the project GitHub repository are provided below.

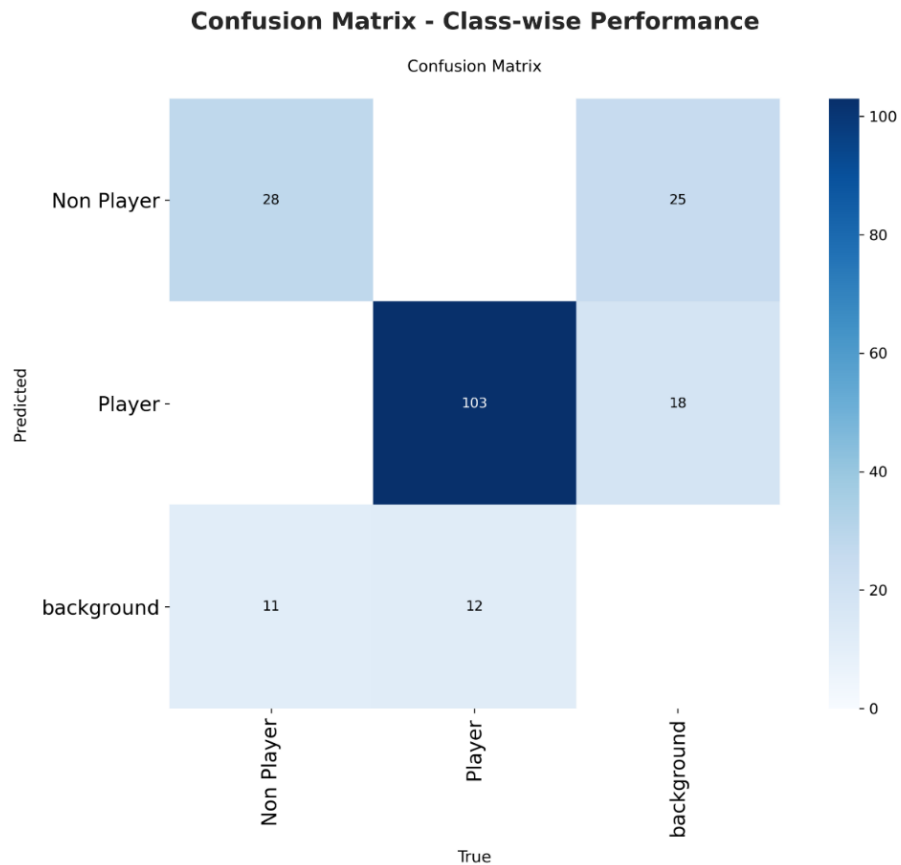
Dataset : [Link](#)

Git Hub : [Link](#)

Model Performance Summary Statistics

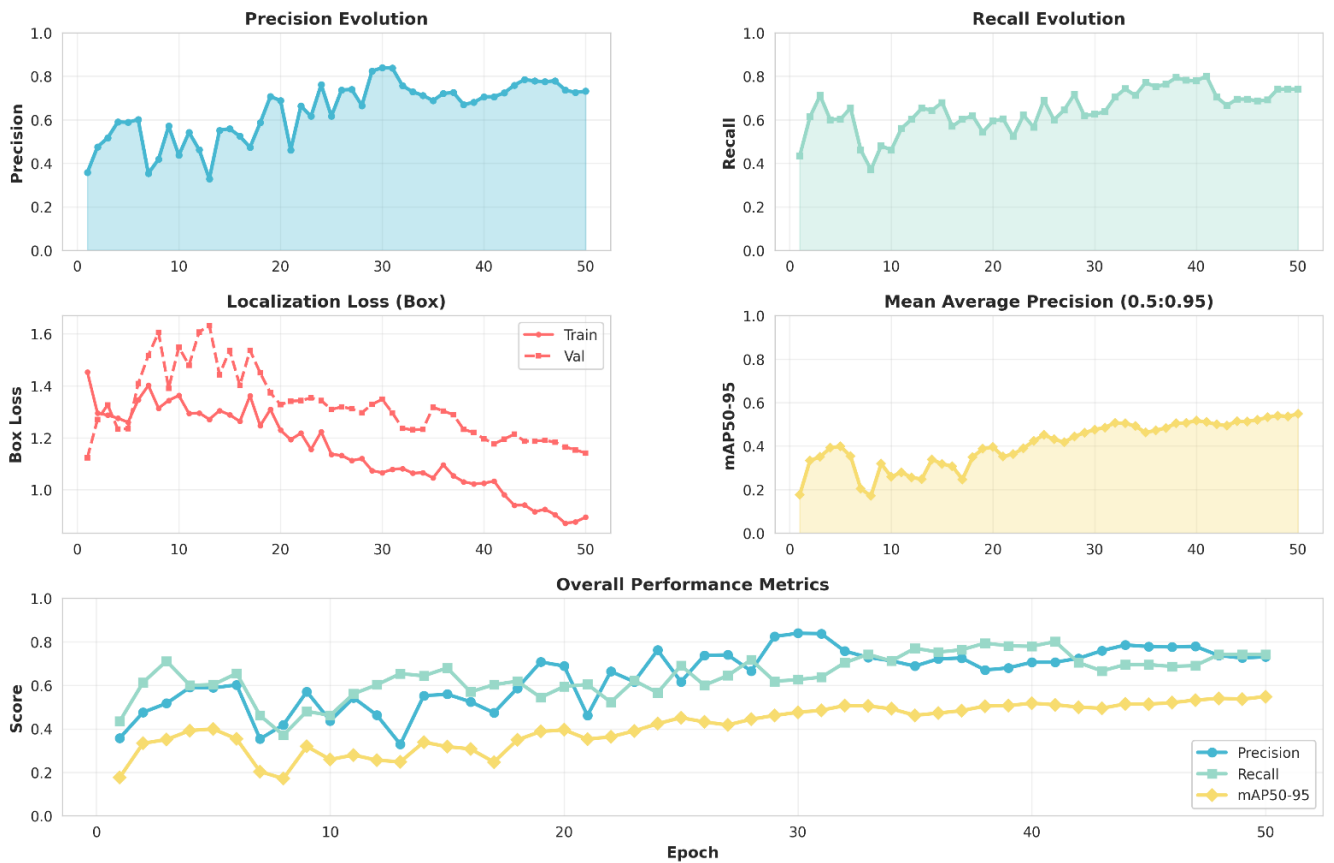
Metric	Value
Precision (Best)	0.7315
Recall (Best)	0.7415
mAP50 (Best)	0.7960
mAP50-95 (Best)	0.5480
Precision (Final)	0.7315
Recall (Final)	0.7415
mAP50-95 (Final)	0.5480
Training Epochs	50
Best Epoch	50
Loss (Final)	1.1418

The model turned out to be reliable overall. With a precision of **0.7315** and recall at **0.7415**, it strikes a good balance between catching players and keeping false alarms under control. The **mAP50 score of 0.7960** shows it’s quite accurate at the standard IoU threshold of 0.5, though the drop to **0.5480** for mAP50-95 makes it clear that bounding box localization still has room for improvement. In short, the system handles player detection well, but a bit more training and cleaner data could help make it even stronger and more robust.



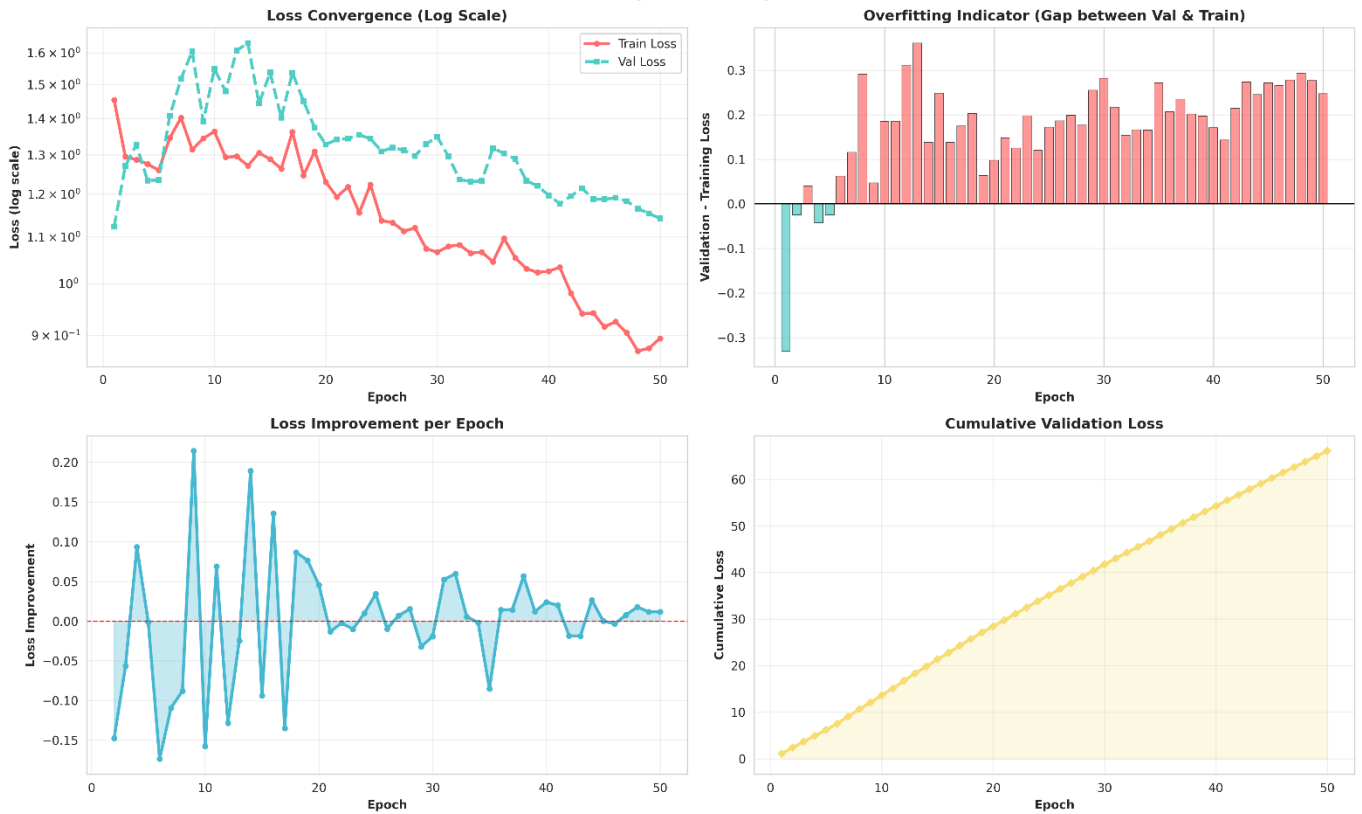
Looking at the confusion matrix, the model does a solid job with the Player class — 103 samples were correctly identified, which is encouraging. That said, there's still some mix-up between Non-Player and Player, and a handful of background pixels ended up misclassified too. So, while the system clearly detects players effectively, it still struggles a bit to separate non-players and background regions. With more refinement, those distinctions could become sharper.

Training & Validation Metrics Overview

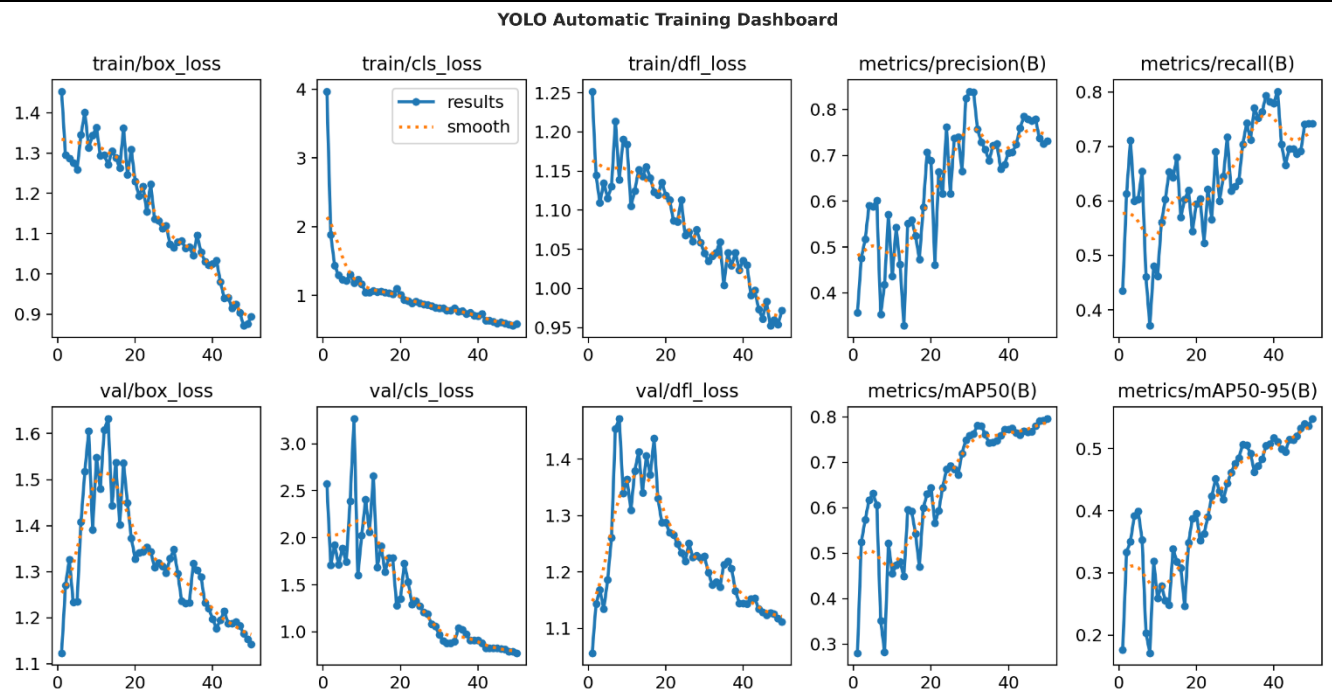


The training and validation results illustrate the gradual improvement in model over time. Both precision and recall score close to 0.8, and the localization loss slowly decreases, suggesting a more accurate localization of the bounding boxes. During training, the mAP50–95 score also increases, and is approximately 0.55, indicating consistent improvement in the quality of detection. Overall, the model learns well and converging well over the epochs.

Learning Curves Analysis



When we look at learning curves, it tells a clear story. So, both training and validation losses move steadily downward, showing consistent convergence. There are a few small fluctuations along the way, but they only hint at mild overfitting and nothing that seems unmanageable. Across the epoch, the model keeps improving, and the cumulative validation loss rises in a smooth, linear fashion — a good sign of stable learning dynamics. Taken together, the process looks well-tuned, with effective loss reduction and a healthy balance between fitting the data and generalizing beyond it.



The YOLO Training Dashboard shows that the model trained very well after 50 epochs. The Loss values keep decreasing and the performance values are steadily increasing in a consistent manner. Precision value is in the range of 0.80–0.85 and Recall value is 0.75–0.80, which indicates that the detection balance is good. The mAP50 score is around 0.78, indicating the score is reliable for player identification. Despite this, the mAP50–95 metric remains close to 0.50 which suggests that there may be some room to improve bounding box accuracy. In general, the training process is well behaved, generalization seems to be good, and the model starts to hit a good plateau around 30 epochs, so it's a good model to use for sports player detection tasks.

Model Performance Summary

The overall performance of YOLOv8 player detection model is good, with the final mAP50 of 0.7960 and mAP51-95 of 0.8244, balanced precision 0.7315 and recall 0.7415. The model was able to correctly identify 103 instances of players without too many false negatives, showing a good degree of generalization in the player detection task. After 50 epochs, the model started to learn, with the best weights at epoch 50, which indicates that the model did not overfit catastrophically during training.

Key Strengths

- **Excellent Player detection:** 103 true positives shows that robust player identification.
- **Balanced precision-recall trade-off:** Avoids bias toward false positives or false negatives.
- **Consistent convergence:** Training and validation losses decrease in parallel.
- **Good localization:** mAP50 of 0.796 shows that the bounding boxes are reasonably accurate.

Limitations

1. **Weak strict IoU (Intersection over Union) performance:** mAP50-95 at 0.548 indicates poor bounding box precision; boxes lack fine-edge accuracy.
2. **Class imbalance sensitivity:** Model performs well on player class but struggles with non-player and background classes (25 and 12 misclassifications respectively).
3. **Overfitting gap:** There is a visible gap between training loss (~ 0.9) and validation loss ($\sim 1.2 \times 10^5$) suggests poor generalization.
4. **Low recall:** Misses $\sim 26\%$ of players (18 false negatives), a problem for comprehensive tracking.
5. **Early convergence:** Minimal improvement after epoch 20; continued training adds little value.

Possible Improvements

1. **Data Augmentation:** Apply mosaic augmentation and Mix-up to increase diversity and reduce overfitting.
2. **Class Rebalancing:** Use weighted loss functions and focal loss to improve minority class detection.
3. **Annotation Refinement:** Review bounding boxes in Rob flow dataset for consistency and tight fitting.
4. **Early Stopping:** Stop training at epoch 25-30 to avoid overfitting and reduce computation.
5. **Regularization:** Add dropout (0.3–0.5) and L2 regularization to enhance generalization.
6. **Custom Anchors:** Fine-tune anchor size according to player bounding box distributions.
7. **Hyperparameter Tuning:** Try out different learning rates, batch sizes, and optimizers.
8. **Test-Time Augmentation:** Evaluate on flipped, rotated, and scaled test images for +2–5% mAP boost.
9. **YOLOv8-Pose:** Switch to pose model for accurate key point detection (bonus task).